Study of resonances in 1-D systems by Smooth Exterior Scaling and Complex Scaling

A B. Tech. Project Report Submitted in Fulfillment of the Requirements for the Degree of

Bachelor of Technology

by

Rajat Patodi
(210122042)

under the guidance of

Dr. Ashish K. Gupta

to the

DEPARTMENT OF CHEMISTRY

INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI
GUWAHATI - 781039, ASSAM

<u>DECLARATION</u>

*I Rajat Patodi,declare that this thesis titled "**Study of resonances in 1-D systems by Smooth Exterior Scaling and Complex Scaling**" is carried out by me in the Department of Chemical Science and Technology, Indian Institute of Technology Guwahati during the period from January-2025 to April-2025 and that it has not been submitted for a degree or any other qualification at this institution or any other university or institution. Where I have consulted the published work of others , this is always clearly attributed. I have acknowledged all the main sources of help. I have not taken any help of large language models (LLMs) for any work related to my project.*

Rajat Patodi

Department of Chemistry

Indian Institute of Technology Guwahati, Assam

April, 2025

CERTIFICATE

*This is to certify that the work contained in this thesis entitled "**Study of resonances in 1-D systems by Smooth Exterior Scaling and Complex Scaling**" is a bonafide work of **Rajat Patodi (Roll No. 210122042)**, carried out in the Department of Chemical Science and Technology, Indian Institute of Technology Guwahati under my supervision and that it has not been submitted elsewhere for a degree.*

Supervisor: **Dr. Ashish K. Gupta**

Department of Chemistry

Indian Institute of Technology Guwahati, Assam

April, 2025

Acknowledgements

I would like to express my sincere gratitude to the Indian Institute of Technology for providing me with the opportunity to work on this project. This research would not have been possible without the support and resources provided by the institution.

I am deeply indebted to my project advisor, Dr. Ashish Kumar Gupta, for his invaluable guidance, expert knowledge, and unwavering encouragement throughout the course of this research. His mentorship has been instrumental in shaping this project and my understanding of the subject matter.

I would also like to extend my heartfelt appreciation to Deepak and Hrishikesh, who played a pivotal role as a helper during this research endeavor. Their dedication and assistance significantly contributed to the successful execution of this project.

I am grateful to all those who have contributed to this work directly or indirectly, providing insights, resources, and support. Your involvement has been invaluable.

Lastly, I want to acknowledge the collective effort of my project partner Raghav, whose collaboration and discussions enriched the research experience.

Contents

List of Figures **iv**

1 Introduction **3**

1.1 Resonance 3

1.2 Complex Scaling 4

2 Computational Details **6**

2.1 One Dimensional Coordinate Transformation of MSES 6

2.2 Complex Scaling 9

3 Observation & Analysis **11**

3.1 Eigenvalue Analysis of original 1d transformation 11

3.1.1 Goal 11

3.1.2 Steps Taken 11

3.1.3 Plot of Results 13

3.1.4 What We See 13

3.2 Python Implementation of 1D Complex Scaling 14

3.2.1 Goal 14

3.2.2 Steps Taken 15

3.2.3 What We See 16

4 Conclusion & Discussion **18**

5 Appendix **19**

5.1 Python Code to find eigen values for 1-d system in case of MSES 19

5.2 Python Code to find eigen values for 1-d system in case of MSES 23

References **27**

List of Figures

3.1 Eigenvalues in the complex plane for different values of θ. 14

3.2 Zoomed-in version showing resonance values @ (0.465,0.004) 14

3.3 Eigenvalues in the complex plane for different values of θ (Python implementation). 17

3.4 Eigenvalues in the complex plane for different values of θ (found at (0.465,0.004)) 17

Abstract

This project builds upon previous work concerning the derivation of Smooth Exterior Scaling in spherical coordinate systems. In the previous semester's study, the results obtained were in close agreement with those reported in the referenced research paper, although some discrepancies were noted. In this continuation, those discrepancies have been addressed and resolved.

The primary objective of this project is to extend the concept of Smooth Exterior Scaling to cylindrical geometries and to generalize the results by employing a methodology analogous to that used in one-dimensional systems.

The findings reported in this study establish a good theoretical basis for future experimental validation and investigation in the domain of quantum chemistry.

Chapter 1

Introduction

1.1 Resonance

Metastable states are quantum states that, while unstable, persist for relatively long periods before decaying into more stable configurations.

In quantum mechanics, these states have finite lifetimes. To analyze these states, physicists often use wave packets—continuous superpositions of quantum states that evolve over time. In spatial terms, metastable states display an *exponential decay*, reflecting their eventual breakdown.

Metastable states play an essential role in various scientific and technological contexts, including:

- High-energy systems such as plasmas and X-ray spectroscopy.
- Electron-molecule collisions relevant in interstellar chemistry.
- DNA damage and radiolysis caused by low-energy (slow) electrons.

Understanding resonance phenomena requires accounting for the finite lifetimes of these metastable states.

These resonance wavefunctions are not square integrable,hence do not belong to the hermitian domain of hamiltonian. Thus, non-hermitian quantum mechanics treatment is required. Some of the methods are "Complex Scaling" and "Smooth Exterior Scaling" [1].

1.2 Complex Scaling

Complex scaling is a powerful technique in quantum mechanics used to analyze unstable or **resonant states**. The core idea involves transforming spatial coordinates into the complex plane. Specifically, this is done by rotating the position vector r by an angle θ:

$$r \to re^{i\theta}$$

Here, θ is a real parameter known as the *scaling angle*.

This transformation modifies the Hamiltonian operator. If $\hat{H}$ is the original Hamiltonian, the scaled Hamiltonian becomes:

$$\hat{H}' = e^{i\theta}\hat{H}e^{-i\theta}$$

Or more generally:

$$\hat{H}(\theta) = \hat{T}(e^{-2i\theta}) + V(re^{i\theta})$$

where $\hat{T}$ is the kinetic energy operator and V is the potential energy.

After applying the transformation, the Schrödinger equation takes the form:

$$\hat{H}(\theta)\Psi^{\theta} = E\Psi^{\theta}$$

Here, Ψ^{θ} is the complex-scaled wavefunction, and E is the corresponding energy.

Why is complex scaling useful?

- **Bound states** retain their original (real) energies.

- **Resonance states** (short-lived, unstable states) now appear as discrete complex energies:

$$E = E_r - \frac{i\Gamma}{2}$$

  where E_r is the resonance energy and Γ is related to the lifetime ($\tau = \hbar/\Gamma$).

- **Continuum states** (unbound) are rotated into the complex energy plane, making resonances easier to identify.

This technique is widely used in atomic, molecular, and nuclear physics to better understand unstable quantum systems.

Chapter 2

Computational Details

2.1 One Dimensional Coordinate Transformation of MSES

We begin with the fundamental Schrödinger equation:

$$H\psi = E\psi \tag{2.1}$$

where the Hamiltonian H is defined by:

$$H = -\frac{\hbar^2}{2m}\frac{d^2}{dz^2} \tag{2.2}$$

For a free particle (no potential) in one dimension:

$$-\frac{\hbar^2}{2m}\frac{d^2}{dz^2}\Psi(z) = E\Psi(z) \tag{2.3}$$

Here, $z = F(x)$ is a path in the complex plane such that:

$$z = F(x) \approx x \cdot e^{i\theta} \quad \text{as } x \to \infty \tag{2.4}$$

We define

$$\frac{dz}{dx} = F'(x) = f(x) = 1 + \left(e^{i\theta} - 1\right) g(x) \tag{2.5}$$

$$g(x) = 1 + \frac{1}{2}\left(\tanh\left(\frac{x - x_0}{\epsilon}\right) - \tanh\left(\frac{x + x_0}{\epsilon}\right)\right) \tag{2.6}$$

where $g(x)$ varies from 0 to 1 around the point $x = x_0$.

Also,

$$\frac{d}{dz} = \frac{1}{f(x)}\frac{d}{dx} \tag{2.7}$$

The volume element in the z-system is given by:

$$dz = f(x)dx \tag{2.8}$$

$$\begin{aligned} \int \psi^*(z)\psi(z)\,dz &= \int \psi^*(z)\psi(z)f(x)\,dx \\ &= \int \left[\psi(z)\sqrt{f(x)}\right]^* \left[\psi(z)\sqrt{f(x)}\right] dx \end{aligned} \tag{2.9}$$

Thus, we define a new wavefunction $\phi(x)$:

$$\phi(x) = \psi(x)\sqrt{f(x)} \tag{2.10}$$

where

$$\psi(x) = \frac{\phi(x)}{\sqrt{f(x)}} \tag{2.11}$$

Now, substitute $\psi(x) = \frac{\phi(x)}{\sqrt{f(x)}}$ and the derivative rules into the Schrödinger equation.

First Derivative

$$\frac{d}{dz}\psi(z) = \frac{1}{f(x)}\frac{d}{dx}\psi(x) \tag{2.12}$$

Second Derivative

$$\frac{d^2}{dz^2}\psi(z) = \frac{1}{f(x)}\frac{d}{dx}\left(\frac{1}{f(x)}\frac{d}{dx}\psi(x)\right) \tag{2.13}$$

Now, substitute $\psi(x) = \frac{\phi(x)}{\sqrt{f(x)}}$ in the Schrödinger equation as well as in the derivative rules:

$$\begin{aligned}\frac{d\psi}{dx} &= \frac{d}{dx}\left(\frac{\phi(x)}{\sqrt{f(x)}}\right)\\ &= \frac{1}{\sqrt{f(x)}}\frac{d\phi}{dx} - \frac{\phi(x)}{2f(x)^{3/2}}\frac{df}{dx}\end{aligned} \tag{2.14}$$

$$\frac{1}{f(x)}\frac{d\psi}{dx} = \frac{1}{f(x)^{3/2}}\frac{d\phi}{dx} - \frac{\phi(x)}{2f(x)^{5/2}}\frac{df}{dx} \tag{2.15}$$

Second Derivative Continued

$$\frac{d}{dx}\left(\frac{1}{f(x)}\frac{d\psi}{dx}\right) = \frac{d}{dx}\left[\frac{1}{f(x)^{3/2}}\frac{d\phi}{dx} - \frac{\phi(x)}{2f(x)^{5/2}}\frac{df}{dx}\right] \tag{2.16}$$

Expanding and simplifying, and then multiplying both sides of the Schrödinger equation by $f(x)^{1/2}$, we eventually arrive at:

$$-\frac{\hbar^2}{2m}\left\{\frac{1}{f(x)}\frac{d^2}{dx^2}\left(\frac{\phi}{f(x)}\right) + \left[-\frac{3}{4}f(x)^{-4}\left(\frac{df}{dx}\right)^2 + \frac{1}{2}f(x)^{-3}\frac{d^2f}{dx^2}\right]\phi\right\} = E\phi \tag{2.17}$$

Final Factored Form and Effective Complex Absorbing Potential

So, the equation becomes:

$$-\frac{\hbar^2}{2m}\left[\frac{1}{f(x)}\frac{d^2}{dx^2}\frac{1}{f(x)} + \left(-\frac{3}{4}\frac{(df/dx)^2}{f(x)^4} + \frac{1}{2}\frac{d^2f/dx^2}{f(x)^3}\right)\right]\phi = E\phi \tag{2.18}$$

where $f(x) = \frac{dz}{dx}$.

Interpretation: Complex Absorbing Potential $V_{CAP}(x)$

The terms

$$-\frac{3}{4}\frac{(df/dx)^2}{f(x)^4}+\frac{1}{2}\frac{d^2f/dx^2}{f(x)^3} \tag{2.19}$$

can be grouped as an **effective potential** (sometimes called the "kinetic potential" or "quantum potential") arising from the transformation of coordinates:

$$V_{CAP}(x)=-\frac{\hbar^2}{2m}\left[-\frac{3}{4}\frac{(df/dx)^2}{f(x)^4}+\frac{1}{2}\frac{d^2f/dx^2}{f(x)^3}\right] \tag{2.20}$$

So, the transformed Schrödinger equation is now read as:

$$-\frac{\hbar^2}{2m}\frac{1}{f(x)}\frac{d^2}{dx^2}\frac{1}{f(x)}\phi(x)+V_{CAP}(x)\phi(x)=E\phi(x) \tag{2.21}$$

This is a standard result in quantum mechanics when changing variables in the Schrödinger equation: the extra term $V_{CAP}(x)$ acts as a complex absorbing potential arising from the coordinate transformation.

2.2 Complex Scaling

1-D Schrödinger Wave Equation

The one-dimensional Schrödinger wave equation is given by:

$$H\psi(x)=E\psi(x) \tag{2.22}$$

where the Hamiltonian H is defined as:

$$H=-\frac{\hbar^2}{2m}\frac{d^2}{dx^2}+V(x) \tag{2.23}$$

We introduce the complex transformation $x \to x' \exp(i\theta)$ such that

$$\begin{aligned} dx &= \exp(i\theta)\, dx' \\ \frac{d}{dx} &= \exp(-i\theta)\frac{d}{dx'} \\ \frac{d^2}{dx^2} &= \exp(-2i\theta)\frac{d^2}{dx'^2} \end{aligned} \tag{2.24}$$

Also, since

$$\int \psi^*(x)\psi(x)\, dx = \int \psi^*(x)\psi(x)\ \exp(i\theta)\, dx' \tag{2.25}$$

one can transform the Hamiltonian in order to simplify the expression of the volume element to be equal to $dx = dx'$ by defining a new function $\phi(x')$ such that

$$\psi(x) = \phi(x') \exp(-0.5i\theta) \tag{2.26}$$

Substituting these values in equation (2.23), we get

$$\begin{aligned} & -\frac{\hbar^2}{2m}\left[\exp(-2i\theta)\frac{d^2}{dx'^2}\right][\exp(-0.5i\theta)\phi(x')] + V\left(x' \exp(i\theta)\right)[\exp(-0.5i\theta)\phi(x')] \\ & = E\left[\exp(-0.5i\theta)\phi(x')\right] \\ \implies \quad & -\frac{\hbar^2}{2m}\exp(-2i\theta)\frac{d^2}{dx'^2}\phi(x') + V\left(x' \exp(i\theta)\right)\phi(x') = E\phi(x') \end{aligned} \tag{2.27}$$

Thus, the complex-scaled Hamiltonian H_θ becomes

$$H_\theta = -\frac{\hbar^2}{2m}\exp(-2i\theta)\frac{d^2}{dx'^2} + V\left(x' \exp(i\theta)\right) \tag{2.28}$$

That is, the transformed Schrödinger equation becomes

$$H_\theta\phi(x') = E\phi(x') \tag{2.29}$$

Chapter 3

Observation & Analysis

3.1 Eigenvalue Analysis of original 1d transformation

3.1.1 Goal

We want to study how the eigenvalues of a Hamiltonian change when we apply a complex path controlled by a parameter θ from 0 to 1.

3.1.2 Steps Taken

Grid Setup

We divide the space from $x = -15$ to 15 into 300 points. The spacing between the points is:

$$\Delta x = \frac{x_{\text{end}} - x_{\text{start}}}{N - 1}$$

Complex Smooth Exterior Path Function

We change the space using a function:

$$f(x, \theta) = 1 + (\exp(i\theta) - 1) \cdot g(x)$$

where

$$g(x) = 1 + \frac{1}{2}\left(\tanh\left(\frac{x - x_0}{\epsilon}\right) - \tanh\left(\frac{x + x_0}{\epsilon}\right)\right)$$

This $g(x)$ controls how the deformation is applied in space.

Kinetic Energy

We use a known formula to build a matrix for kinetic energy:

$$T_{ij} = \begin{cases} \frac{\hbar^2\pi^2}{6m\Delta x^2} & \text{if } i = j \\ \frac{\hbar^2(-1)^{i-j}}{m\Delta x^2(i-j)^2} & \text{if } i \neq j \end{cases}$$

Extra Term

Finally we use $V_{CAP}(x)$ as follows:

$$V_{CAP}(x) = \frac{1}{8m} \cdot \left(\frac{2f''(x)f(x) - 3(f'(x))^2}{f(x)^4}\right)$$

Potential Function

We use a potential:

$$V_F(x) = \left(1 - \frac{V_0}{\cosh^2(Bx)}\right) \exp\left(-a\,x^2\right)$$

Hamiltonian and Eigenvalues

We combine all parts into one Hamiltonian:

$$H_\theta = MTM + V_k + V_F$$

where:

- $M = \mathrm{diag}\left(\frac{1}{f(x_i,\theta)}\right)$
- V_{CAP} is a diagonal matrix made from $V_{CAP}(x)$

- V_F is a diagonal matrix made from $V_F(x)$

We then find the eigenvalues of H_θ using a numerical solver.

3.1.3 Plot of Results

We calculate the eigenvalues for many values of θ between 0 and 1. We then plot them in the complex plane — real part on the x-axis and imaginary part on the y-axis. Color shows the value of θ.

3.1.4 What We See

From Figure 3.1, we see eigenvalues with variation of θ, and from Figure 3.2 we see the exact resonance value that matches the one given by the research article by O.Karlsson.

$$\text{Resonance value obtained at : } 0.465 + i0.004$$

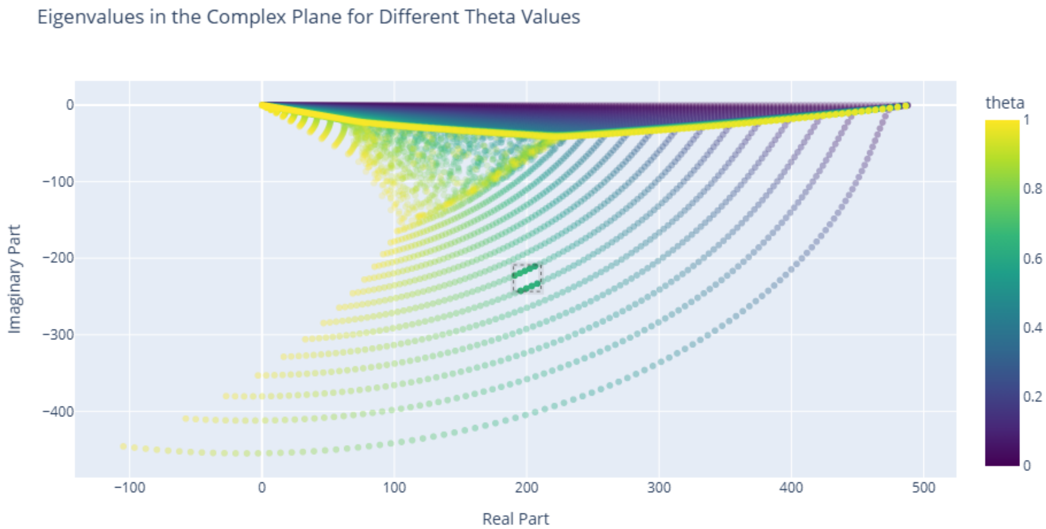

Fig. 3.1 Eigenvalues in the complex plane for different values of θ.

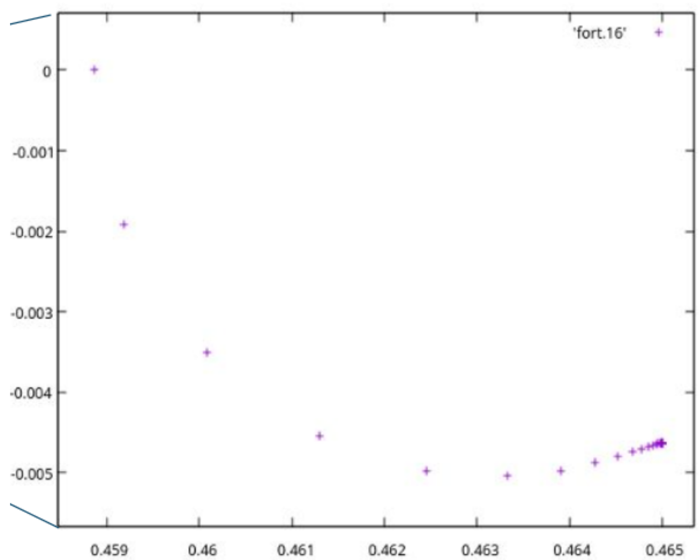

Fig. 3.2 Zoomed-in version showing resonance values @ (0.465,0.004)

3.2 Python Implementation of 1D Complex Scaling

3.2.1 Goal

We aim to numerically explore how the eigenvalues of a 1D quantum Hamiltonian evolve under complex scaling, controlled by a parameter θ. This approach helps us reveal resonance states by rotating the continuum spectrum into the complex plane.

3.2.2 Steps Taken

Grid Setup

We define a 1D spatial grid:

$$x \in [x_{\text{start}}, x_{\text{end}}] = [-15, 15]$$

with $N = 300$ points, giving a spacing:

$$\Delta x = \frac{x_{\text{end}} - x_{\text{start}}}{N - 1}$$

This grid is stored in the array `x_values`.

Kinetic Energy Matrix

The kinetic energy operator is discretized using a well-known formula:

$$T_{ij} = \begin{cases} \frac{\hbar^2 \pi^2}{6m\Delta x^2} & \text{if } i = j \\ \frac{\hbar^2 (-1)^{i-j}}{m\Delta x^2 (i-j)^2} & \text{if } i \neq j \end{cases}$$

This is implemented as a complex-valued $N \times N$ matrix `T`.

Potential Function with Complex Scaling

The potential is defined as:

$$V_F(x, \theta) = \left(1 - \frac{1}{\cosh^2(xe^{i\theta})}\right) \exp\left(-0.05\, x^2 e^{2i\theta}\right)$$

This function is evaluated at each grid point for every value of θ.

Complex Scaling Transformation

Complex scaling is introduced by multiplying the kinetic energy operator by $\exp(-2i\theta)$, effectively rotating the continuum spectrum in the complex plane.

Hamiltonian Construction

For each value of θ:

$$H_\theta = T \cdot e^{-2i\theta} + V_F(x, \theta)$$

where T is the kinetic matrix and V_F is a diagonal matrix containing the potential at each grid point.

Eigenvalue Computation

For each θ in a range from 0 to 1 (101 steps), we:

- Construct the Hamiltonian H_θ
- Compute all eigenvalues using `scipy.linalg.eigvals`
- Store the real and imaginary parts of each eigenvalue, along with the corresponding θ

3.2.3 What We See

The resulting plot (Figure 3.3) shows how the eigenvalues move in the complex plane as θ increases. Resonance states appear as isolated points that remain stationary as θ varies, while the continuum rotates downward. This matches the physical expectation and provides a numerical confirmation of the complex scaling method.

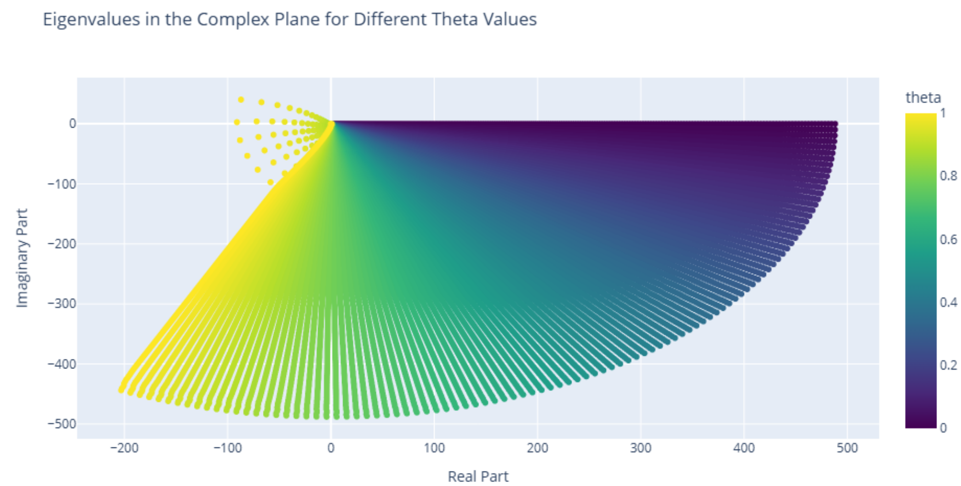

Fig. 3.3 Eigenvalues in the complex plane for different values of θ (Python implementation).

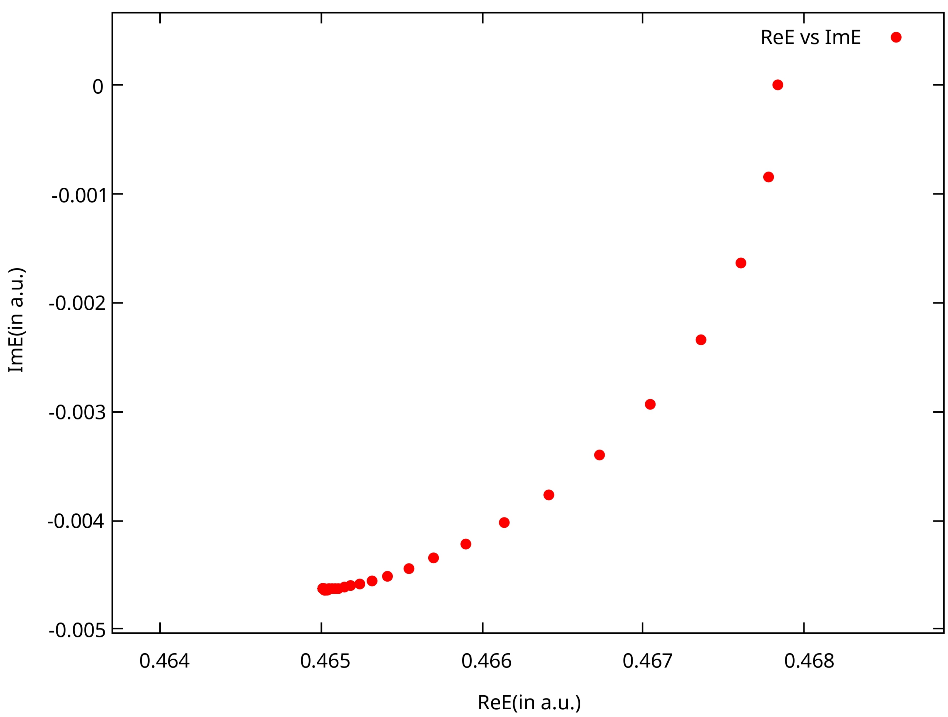

Fig. 3.4 Eigenvalues in the complex plane for different values of θ (found at (0.465,0.004))

.

Chapter 4

Conclusion & Discussion

Through this project that was built on top of the previous project by continuing the previous work, errors in the one-dimensional transformation were successfully resolved, and the results obtained aligned with those shown in O.Karlsson's paper [2], that is,

$$V_{CAP}(x) = -\frac{\hbar^2}{2m}\left[-\frac{3}{4}\frac{(df/dx)^2}{f(x)^4} + \frac{1}{2}\frac{d^2f/dx^2}{f(x)^3}\right]$$

We further applied the transformation for complex scaling in 1-D coordinate system and subsequently plotted the eigenvalues for the final results.

Using results from both methods and the potential function mentioned in the article by Ido Gilary [3], first resonance values were obtained at the same position: $0.465 + i0.004$.

Future work will revolve around understanding the eigenvalues plot, confirming the results obtained, and subsequently using the transformed Hamiltonian for new quantum systems.

Chapter 5

Appendix

5.1 Python Code to find eigen values for 1-d system in case of MSES

```
[language=Python]
import numpy as np
import pandas as pd
import plotly.express as px
from scipy.linalg import eigvals

# Constants
h_bar = 1.0
mass = 1.0
N = 300 # Matrix size
x_start, x_end = -15, 15
num_points = N
x_values = np.linspace(x_start, x_end, num_points)
delta_x = (x_end - x_start) / (num_points - 1)
epsilon_value = 1e-8 # Small epsilon to avoid division by zero
```

```
# Define g(x) using the new formula
def g(x, x0=12.0, epsilon=2):
    return 1 + 0.5 * (np.tanh((x - x0) / epsilon) - np.tanh((x + x0) / epsilon))

# Define f(x) using g(x)
def f(x, theta):
    return 1 + (np.exp(1j * theta) - 1) * g(x) # Adjust f(x) to include theta

# First partial derivative of f(x) with respect to x
def partial_derivative_f(x, theta, x0=12.0, epsilon=0.1):
    factor = np.exp(1j * theta) - 1
    g_prime = 0.5 * ((1 / np.cosh((x - x0) / epsilon))**2 - (1 / np.cosh((x + x0) /
        ↪ epsilon))**2) * (1 / epsilon)
    return factor * g_prime

# Second partial derivative of f(x) with respect to x
def second_partial_derivative_f(x, theta, x0=12.0, epsilon=0.1):
    factor = np.exp(1j * theta) - 1
    sech2_1 = (1 / np.cosh((x - x0) / epsilon))**2
    sech2_2 = (1 / np.cosh((x + x0) / epsilon))**2
    tanh1 = np.tanh((x - x0) / epsilon)
    tanh2 = np.tanh((x + x0) / epsilon)
    g_double_prime = 0.5 * (-2 * sech2_1 * tanh1 + 2 * sech2_2 * tanh2) * (1 /
        ↪ epsilon**2)
    return factor * g_double_prime

# Define T matrix using the provided formula
T = np.zeros((N, N), dtype=np.complex128)
for i in range(N):
    for j in range(N):
        if i == j:
```

```
            T[i, j] = (h_bar**2 * (-1)**(i - j) * np.pi**2) / (3 * 2 * mass * delta_x
                ↪ **2)
        else:
            T[i, j] = (h_bar**2 * (-1)**(i - j)) / (mass * delta_x**2 * (i - j)**2)



# Define F(x) function
lambda_ = 1.0 # Example lambda value
epsilon = 2 # Example epsilon value
x0 = 12.0 # Example x0 value

def F(x, lambda_=lambda_, epsilon=epsilon, x0=x0):
    return x + lambda_ * (x + np.log(np.cosh((x - x0) / epsilon) / np.cosh((x + x0) /
        ↪  epsilon)))

# Construct V_k(x) using the definition provided
def V_k(x, theta, mass=mass):
    f_x = f(x, theta)
    f_prime = partial_derivative_f(x, theta)
    f_double_prime = second_partial_derivative_f(x, theta)
    return (1 / (8 * mass)) * ((2 * f_double_prime * f_x - 3 * f_prime**2) / (f_x**4)
        ↪ )

def V_F(x):
    return np.array( (1 - 1 / (np.cosh(1 * x) * np.cosh(1 * x))) * np.exp(-0.05 * x *
        ↪  x))

# Construct Vk and F matrices independently of theta
Vk = np.diag(V_k(x_values, theta=0)) # Diagonal matrix with V_k(x_i), initialized
    ↪ with theta=0
VF = np.diag(V_F(x_values)) # Diagonal matrix with V(F(x_i))
```

```
# Range of theta values
theta_values = np.linspace(0, 1, 101) # 101 values from 0 to 1

# Prepare data for saving and plotting
data_to_save = []

# Loop over theta values, construct Hamiltonian, and compute eigenvalues
for theta in theta_values:
    # Construct diagonal matrix M using f(x) with current theta
    M = np.diag(1 / (f(x_values, theta))) # Diagonal matrix with 1/f(x_i)

    # Construct Hamiltonian H = M * T * M + Vk + VF
    H = M @ T @ M + Vk + VF

    # Solve for eigenvalues of the Hamiltonian H
    eigenvalues = eigvals(H)

    # Store theta and eigenvalues in data
    for ev in eigenvalues:
        data_to_save.append([theta, np.real(ev), np.imag(ev)])

# Save data to a text file
output_file = 'eigenvalues_data_theta.txt'
with open(output_file, 'w') as f:
    f.write('theta\treal\timag\n')
    for row in data_to_save:
        f.write(f"{row[0]}\t{row[1]}\t{row[2]}\n")

print(f"Data saved to {output_file}")
```

```
# Convert data to DataFrame for easy plotting with Plotly
df = pd.DataFrame(data_to_save, columns=['theta', 'real', 'imag'])


# Interactive plot with Plotly
fig = px.scatter(df, x='real', y='imag', color='theta', color_continuous_scale='
    ↪ Viridis',
                title='Eigenvalues in the Complex Plane for Different Theta Values',
                labels={'real': 'Real Part', 'imag': 'Imaginary Part'})
fig.update_layout(xaxis_title="Real Part", yaxis_title="Imaginary Part")
fig.show()
```

5.2 Python Code to find eigen values for 1-d system in case of MSES

```
[language=Python]
import numpy as np
import pandas as pd
import cmath
import plotly.express as px
from scipy.linalg import eigvals


# Constants
h_bar = 1.0
mass = 1.0
N = 300 # Matrix size
x_start, x_end = -15, 15
num_points = N
x_values = np.linspace(x_start, x_end, num_points)
delta_x = (x_end - x_start) / (num_points - 1)
epsilon_value = 1e-8 # Small epsilon to avoid division by zero
```

```
# Define T matrix using the provided formula
T = np.zeros((N, N), dtype=np.complex128)
for i in range(N):
    for j in range(N):
        if i == j:
            T[i, j] = (h_bar**2 * (-1)**(i - j) * np.pi**2) / (3 * 2 * mass * delta_x
                ↪ **2)
        else:
            T[i, j] = (h_bar**2 * (-1)**(i - j)) / (mass * delta_x**2 * (i - j)**2)

# Define F(x) function
lambda_ = 1.0 # Example lambda value
epsilon = 2 # Example epsilon value
x0 = 12.0 # Example x0 value

def V_F(x,theta):
    return np.array( (1 - 1 / (np.cosh(1 * x*(cmath.exp(1j * theta))) * np.cosh(1 * x
        ↪ *(cmath.exp(1j * theta))))) * np.exp(-0.05 * x * x*(cmath.exp(2j * theta))
        ↪ ))

# Construct Vk and F matrices independently of theta
# VF = np.diag(V_F(x_values)) # Diagonal matrix with V(F(x_i))

# Range of theta values
theta_values = np.linspace(0, 1, 101) # 101 values from 0 to 1

# Prepare data for saving and plotting
data_to_save = []
```

```
# Loop over theta values, construct Hamiltonian, and compute eigenvalues
for theta in theta_values:
    # Construct diagonal matrix M using f(x) with current theta
    VF = np.diag(V_F(x_values,theta))
    # M = np.diag(1 / (f(x_values, theta))) # Diagonal matrix with 1/f(x_i)

    # Construct Hamiltonian H = M * T * M + VF
    H = T*(cmath.exp(-2j * theta)) + VF

    # Solve for eigenvalues of the Hamiltonian H
    eigenvalues = eigvals(H)

    # Store theta and eigenvalues in data
    for ev in eigenvalues:
        data_to_save.append([theta, np.real(ev), np.imag(ev)])

# Save data to a text file
output_file = 'eigenvalues_data_theta.txt'
with open(output_file, 'w') as f:
    f.write('theta\treal\timag\n')
    for row in data_to_save:
        f.write(f"{row[0]}\t{row[1]}\t{row[2]}\n")

print(f"Data saved to {output_file}")

# Convert data to DataFrame for easy plotting with Plotly
df = pd.DataFrame(data_to_save, columns=['theta', 'real', 'imag'])

# Interactive plot with Plotly
fig = px.scatter(df, x='real', y='imag', color='theta', color_continuous_scale='
    ↪ Viridis',
```

```
                title='Eigenvalues in the Complex Plane for Different Theta Values',
                labels={'real': 'Real Part', 'imag': 'Imaginary Part'})
fig.update_layout(xaxis_title="Real Part", yaxis_title="Imaginary Part")
fig.show()
```

References

[1] Nimrod Moiseyev. Quantum theory of resonances: calculating energies, widths and cross-sections by complex scaling. *Physics Reports*, 302(5):212–293, 1998.

[2] Hans O. Karlsson. Accurate resonances and effective absorption of flux using smooth exterior scaling. *The Journal of Chemical Physics*, 109(21):9366–9371, 12 1998.

[3] Shachar Klaiman and Ido Gilary. Chapter 1 - on resonance: A first glance into the behavior of unstable states. In Cleanthes A. Nicolaides, Erkki Brändas, and John R. Sabin, editors, *Advances in Quantum Chemistry*, volume 63 of *Advances in Quantum Chemistry*, pages 1–31. Academic Press, 2012.